

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

MedAccess

A Pharmacist-Supervised Platform for Medicine Availability, Price Comparison and Safe Brand Matching Using the Common Drug Codes for India

Project Report

submitted for the course

BCSE301L — Software Engineering

Fall Semester 2026–27

by

Name	Registration Number
Bhuvanesh Jatla	24BYB1113
Akshit Sitaram	24BYB1111
Shailesh P	24BYB1108
Rishabh Sharma	23BRS1369

Vellore Institute of Technology, Chennai

November 2026

Abstract

Patients in India frequently cannot tell which nearby pharmacy stocks a prescribed medicine, what it costs there, or whether a cheaper brand with the same composition exists. Consumer applications that answer these questions tend to present lower-priced alternatives as interchangeable without professional oversight.

This report describes **MedAccess**, the software project we built for BCSE301L Software Engineering. MedAccess is a role-based web platform that combines the national Common Drug Codes for India (CDCI) catalogue of 93,911 brands with pharmacy-reported availability and indicative prices, and that identifies *potential* brand matches sharing an identical CDCI generic identifier. A match is treated as informational only until a pharmacist explicitly verifies or rejects it through an audited state machine. Controlled and narcotic substances are excluded when the search index is built, by a reviewable, whole-word restricted-substance list, so that no role or route can return them. Every stock, price, verification and administrative action is attributed, time-stamped and recorded in an audit log, and stale availability data is flagged rather than presented as current.

The system supports six roles (patient, caregiver, pharmacist, pharmacy staff, administrator and doctor). It is implemented in Node.js with no third-party runtime dependencies, an in-memory search index and a modular route architecture. Evaluation with 95 automated tests, including real-browser and accessibility audits, shows median search latency of 33–77 ms over 90,939 indexed brands, complete exclusion of 2,972 restricted brands (3.2 %), and full or partial satisfaction of all 39 functional requirements of the software requirements specification.

The complete source code, database and instructions are available at `https://github.com/bhuvaneshjatla-cpu/MedAccess`, and Appendix A lists the demo accounts for every role.

Keywords: medicine availability, price transparency, drug terminology, pharmacist verification, role-based access control, health information systems, audit logging

Contents

1 Introduction **3**
1.1 Background . . . 3
1.2 Problem Statement . . . 3
1.3 Objectives . . . 3
1.4 Scope . . . 3
1.5 Contributions . . . 3
1.6 Organisation of the Report . . . 4

2 Literature Review **4**
2.1 Medicine Prices and Availability . . . 4
2.2 Drug Terminology and Catalogues . . . 4
2.3 Generic Medicine Programmes and Consumer Applications . . . 4
2.4 Access Control and Accountability . . . 4

3 Requirements Analysis **5**
3.1 Software Engineering Process . . . 5
3.2 Stakeholders and Elicitation . . . 5
3.3 Functional Requirements . . . 5
3.4 Business Rules . . . 5
3.5 Non-Functional Requirements . . . 5

4 System Design **6**
4.1 Architecture Overview . . . 6
4.2 Request Pipeline and Security . . . 6
4.3 Data Stores . . . 6
4.4 Catalogue Indexing and Restricted Medicines . . . 6
4.5 Search, Attribute Parsing and Filters . . . 7
4.6 Potential Matching . . . 7
4.7 Pharmacist Verification Workflow . . . 7
4.8 Availability, Prices and Freshness . . . 8
4.9 Reports, Reminders and Audit . . . 8

5 Implementation **8**
5.1 Technology Stack . . . 8
5.2 Modularity . . . 8
5.3 Data . . . 9
5.4 Source Code Repository . . . 9

6 Demonstration: How the System Works **9**
6.1 Patient and Caregiver . . . 9
6.2 Pharmacist . . . 11
6.3 Pharmacy Staff . . . 11
6.4 Administrator . . . 12
6.5 Access Control, Mobile and Dark Mode . . . 13

7 Testing and Evaluation **13**
7.1 Test Methodology . . . 13
7.2 Performance . . . 14
7.3 Safety Validation . . . 14
7.4 Requirements Coverage . . . 14

8 Discussion and Limitations **15**

9 Conclusion and Future Work **15**

A Running the System and Demo Accounts **15**
A.1 Download . . . 15
A.2 Run . . . 16
A.3 Demo Accounts . . . 16

References **16**

1 Introduction

1.1 Background

Access to essential medicines depends not only on whether a medicine exists in the market but on whether a patient can find it at an affordable price in a nearby pharmacy. Cross-national surveys have repeatedly reported low public-sector availability and high, widely varying private-sector prices for essential medicines in low- and middle-income countries [1, 2]. In India, the same molecule at the same strength is typically marketed under many brand names by different manufacturers, with substantial price differences between brands; the Government's Jan Aushadhi programme was created precisely to make equivalent generic medicines available at lower cost [3].

1.2 Problem Statement

Two practical information gaps follow from this situation. First, a patient or caregiver rarely knows which pharmacy currently stocks a medicine and at what price. Second, even when a lower-priced brand with the same composition exists, the patient has no safe way to establish that it is a suitable substitute. Ingredient name alone is insufficient: strength, dosage form, release mechanism and pack size all matter, and substitution decisions are a professional responsibility. Consumer tools that list "cheaper alternatives" automatically risk presenting a price comparison as a clinical equivalence claim.

1.3 Objectives

The objectives of the project were to:

1. let patients and caregivers search the national medicine catalogue and see, for each brand, which pharmacies stock it and at what indicative price, with the source and age of every value;
2. identify other brands with the same composition as *potential* matches, while never presenting them as interchangeable;
3. route every requested comparison to a pharmacist, who alone can verify or reject it;
4. keep controlled, narcotic and psychotropic medicines completely out of the system;
5. give pharmacy staff and administrators the tools to keep data current and to review every change through an audit log; and
6. follow a disciplined software engineering process: feasibility study, requirements elicitation, a software requirements specification (SRS), UML design, implementation and automated testing.

1.4 Scope

MedAccess is an information assistant. It does *not* diagnose, prescribe, change doses or recommend substitution, and it does not sell, take payment for or deliver medicines. All pharmacy, stock and price data in the prototype is synthetic and clearly labelled as such.

1.5 Contributions

The main contributions of the project are:

1. A catalogue-grounded matching method that proposes potential brand matches only when two brands share an identical CDCI generic identifier (same ingredients, strength and dosage form), and that exposes the remaining differences (manufacturer, pack size, price per unit) instead of hiding them.
2. An explicit pharmacist-verification workflow, modelled as a state machine with attributed, time-stamped and audited transitions, in which no comparison can become "verified" without a human decision.

3. A fail-closed exclusion mechanism for controlled and narcotic substances, applied once when the search index is built, so that excluded brands are unreachable through every role and every API route.
4. A provenance-aware availability and price model that merges read-only seed data with attributed staff updates and flags stale records, together with a modular, dependency-free implementation verified by 95 automated tests.

1.6 Organisation of the Report

Section 2 reviews related work. Section 3 summarises the requirements. Section 4 presents the system design and its core mechanisms, and Section 5 the implementation. Section 6 is a screen-by-screen demonstration of how the system works for every role. Section 7 reports testing and evaluation, Section 8 discusses limitations, and Section 9 concludes. Appendix A explains how to download, run and log in to the system.

2 Literature Review

2.1 Medicine Prices and Availability

The WHO/Health Action International methodology [2] standardised the measurement of medicine prices, availability and affordability through facility surveys, and its application across 36 countries documented poor availability and large price mark-ups [1]. These studies establish the problem but are periodic surveys; they do not provide patients with current, pharmacy-level information. MedAccess addresses the latter while keeping the provenance and age of each data point visible.

2.2 Drug Terminology and Catalogues

Reliable comparison requires a controlled vocabulary in which a brand is linked to a generic concept with a defined strength and dosage form. The Common Drug Codes for India (CDCI), maintained by the National Resource Centre for EHR Standards (NRCeS), provides national identifiers for generics, products, brands and suppliers [4]. MedAccess uses CDCI as its authoritative reference and deliberately limits automatic matching to the generic identifier that CDCI itself assigns, rather than to free-text name similarity.

2.3 Generic Medicine Programmes and Consumer Applications

Government initiatives such as Pradhan Mantri Bhartiya Janaushadhi Pariyojana promote quality generic medicines at reduced prices [3]. Commercial e-pharmacy and price-comparison applications commonly list lower-priced substitutes, but they couple discovery with sales and seldom record who, if anyone, confirmed that a substitute is appropriate. MedAccess intentionally excludes sales, payment and delivery, and separates informational matching from professional verification.

2.4 Access Control and Accountability

Role-based access control (RBAC) [5, 6] is the established model for health information systems in which permissions follow professional roles. Accountability additionally requires that changes be attributable and auditable, and common web-application risks such as broken access control and injection remain the leading vulnerability classes [7]. MedAccess applies route-level RBAC with per-request account-status checks, parameterised SQL and a before/after audit trail. Synthetic data is used throughout, following the practice of using realistic synthetic records to develop health software without exposing real patients [8].

3 Requirements Analysis

3.1 Software Engineering Process

The project followed the phases taught in the course. A feasibility study and literature review were followed by requirements elicitation, a requirements analysis document and a Software Requirements Specification structured according to ISO/IEC 29148 [9]. The design was then modelled in UML (use-case, activity, sequence, state, entity–relationship and architecture diagrams), implemented incrementally, and verified with automated tests traced back to the SRS. All of these documents are included in the `docs/` folder of the project repository.

3.2 Stakeholders and Elicitation

Structured questionnaires were prepared for seven stakeholder groups (patients, caregivers, doctors, pharmacists, pharmacy staff, pharmacies and administrators). For this academic prototype the responses were simulated from the perspective of each group, and the resulting needs were consolidated into the SRS. The SRS defines 39 functional requirements (FR), 18 non-functional requirements (NFR) and 10 business rules (BR).

3.3 Functional Requirements

The functional requirements group into nine areas: authentication and roles; medicine search and details; pharmacy availability; indicative prices and comparison; potential matching and pharmacist verification; stock and price updates; incorrect-information reports; refill reminders; and administration (users, catalogue, reports and audit). Table 1 summarises the capabilities granted to each of the six roles.

Table 1: Role–capability matrix

Capability	Patient	Caregiver	Pharmacist	Staff	Admin	Doctor
Search catalogue / details	✓	✓	✓	✓	✓	✓
View availability and prices	✓	✓	✓	✓	✓	–
View potential matches	✓	✓	✓	–	–	✓
Request pharmacist verification	✓	✓	✓	–	–	–
Verify / reject a comparison	–	–	✓	–	–	–
Update stock and price	–	–	✓	✓	–	–
Submit incorrect-info report	✓	✓	✓	✓	–	–
Review and resolve reports	–	–	✓	✓	✓	–
Refill reminders	✓	✓	–	–	–	–
Users, catalogue, reports, audit	–	–	–	–	✓	–

3.4 Business Rules

Four business rules shape the design more than any other requirement. A potential match is informational only (BR-01); only a pharmacist may verify it (BR-02); a lower price does not imply suitability (BR-03); and controlled, narcotic and psychotropic medicines are outside the scope of the system (BR-07), consistent with their special regulatory status under the Narcotic Drugs and Psychotropic Substances Act [10]. The system also performs no diagnosis or prescribing (BR-06) and no sales, payment or delivery (BR-08).

3.5 Non-Functional Requirements

Key NFRs are server-side authorisation (NFR-01–04), minimal personal data and synthetic demonstration data (NFR-06, NFR-07), mobile-friendly and accessible presentation (NFR-09, NFR-11), source and time on every availability and price value (NFR-13), never presenting stale data as current (NFR-14), traceability of changes (NFR-16) and a modular structure (NFR-18). Accessibility was assessed against WCAG 2.1 success criteria [11].

4 System Design

4.1 Architecture Overview

MedAccess follows a three-tier architecture (Fig. 1). A single-page client built from plain ES modules communicates with a JSON API over HTTP. The server is organised as a composition root that builds a shared *kernel* (database helpers, audit writer, route registry, role constants and reference stores) and registers ten component modules, one per functional area, each declaring its routes as `route(method, path, roles, handler)`. Data is held in four stores: an application database, the read-only CDCI catalogue, a read-only synthetic inventory, and an in-memory search index derived from the catalogue.

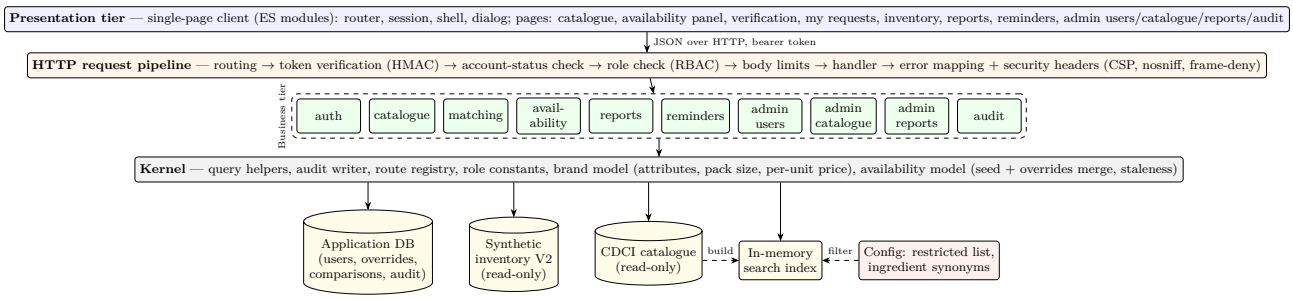


Figure 1: Layered architecture of MedAccess

4.2 Request Pipeline and Security

Every request passes through the same pipeline. Bearer tokens are HMAC-signed [12] and valid for eight hours, and the account status is re-read from the database on every request so that deactivating an account takes effect immediately. Each route declares the roles allowed to call it, so authorisation is enforced on the server rather than by hiding user-interface elements. Passwords are hashed with scrypt [13], login attempts are throttled after five failures, all SQL statements are parameterised, and responses carry Content-Security-Policy, `nosniff` and frame-deny headers, addressing the OWASP classes of broken access control and injection [7]. Every login, denied access and data change is written to the audit log with actor, time, affected record and, for updates, the before and after values.

4.3 Data Stores

The application database (SQLite [14]) holds users and roles, staff stock and price updates, catalogue overlay entries, comparison records and comments, reports, reminders and the audit log. The CDCI catalogue and the synthetic inventory are opened read-only; the application never writes to them. Staff updates are therefore stored as *overrides* that are merged with the seed at read time (Section 4.8), and administrator corrections to the catalogue are stored as an *overlay* that can hide or annotate a brand, record its pack size, or add a brand that CDCI does not list.

4.4 Catalogue Indexing and Restricted Medicines

Fig. 2 shows how the catalogue becomes searchable. At start-up each CDCI brand row is joined with its generic and dosage-form master records and tested against the restricted list L, which contains 65 substances and a classification rule. Let $\mathrm{tok}(b)$ be the set of word tokens in the generic, product and brand names of brand b, and $\mathrm{cls}(b)$ its CDCI classification. The excluded set is

$$R = \{\, b \mid \exists s \in L :\ \mathrm{w}(s) \sqsubseteq \mathrm{tok}(b) \,\} \ \cup\ \{\, b \mid \mathrm{cls}(b) \in K \,\}, \tag{1}$$

where $\mathrm{w}(s) \sqsubseteq \mathrm{tok}(b)$ means that the words of substance s occur contiguously among the tokens of b, and K is the set of restricted classifications. Brands in R are never inserted into the index, so no route can return them. Matching is whole-word: *morphine* excludes morphine products but not *apomorphine*, a non-narcotic dopamine agonist. If the list is missing or malformed the server refuses to start (fail-closed).

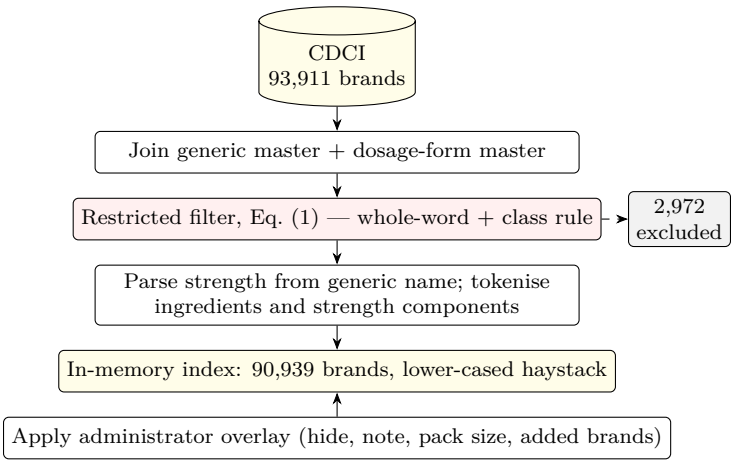


Figure 2: Start-up pipeline that builds the searchable catalogue

4.5 Search, Attribute Parsing and Filters

A substring scan with SQL `LIKE` over the joined catalogue view took about 0.67 s per query on the test machine, so the searchable columns are copied once into an in-memory table with a lower-cased haystack and queried with `instr()`. Users may combine free text with separate active-ingredient, strength and dosage-form filters. Dosage form is taken from CDCI's structured form master; strength is parsed from the generic name and is labelled as unverified. A strength filter such as "50" matches only the numeric component of a strength token, so that it finds 50 mg/5 mL but not 500 mg. A reviewable synonym list (e.g., paracetamol and acetaminophen) widens ingredient search without affecting matching.

4.6 Potential Matching

For a selected brand b with CDCI generic identifier $g(b)$, the set of potential matches is

$$M(b) = \{\, b' \in C \setminus (R \cup H) \mid g(b') = g(b),\ b' \neq b \,\}, \tag{2}$$

where C is the catalogue and H the set of brands hidden by an administrator. Because a CDCI generic identifier encodes ingredients, strength and dosage form together, members of $M(b)$ share all three by construction. For each match the system reports what still differs: brand, manufacturer and pack size (same, differs or unknown), together with the lowest demonstration price and, when the pack size begins with a unit count, an approximate price per unit.

4.7 Pharmacist Verification Workflow

A match carries no claim of equivalence until a user requests verification. The request creates a comparison record whose life cycle follows the state machine in Fig. 3, modelled after the statechart notation [15]. Only a pharmacist can start a review, verify or reject, and each terminal transition records the reviewer, the time and an optional comment. A rejected pair may be requested again, creating a new record and preserving the history. The requester follows progress on a *My verification requests* page.

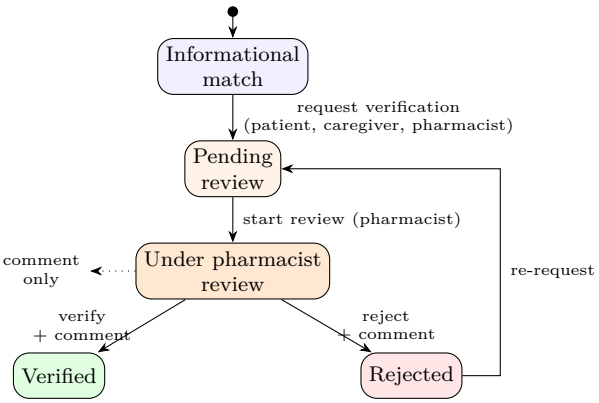


Figure 3: State machine of a brand comparison

4.8 Availability, Prices and Freshness

Availability and price for a brand at a pharmacy are obtained by merging the read-only seed row with the newest staff override for that pair; the override wins and carries the updater's name, the time and the source label. Prices are stored as integer paise to avoid rounding errors and returned in rupees. A record r with update time t_r is flagged stale when

$$\text{stale}(r) = \left[\, t_{\text{now}} - t_r > \tau \,\right], \qquad \tau = 168 \text{ h}, \tag{3}$$

and is then shown with a "Possibly out of date" badge instead of being presented as current (NFR-14). Because the synthetic seed has fixed timestamps, every seed row would eventually be stale. The demonstration therefore shifts each seed time by $\Delta = T_0 - \max_r t_r$, where T_0 is the server start time, which keeps the relative spread of the data while anchoring the newest row to the present. The shift is disclosed in the source label and can be disabled.

4.9 Reports, Reminders and Audit

Any patient, caregiver, pharmacist or staff member can report an incorrect price or availability for a specific brand and pharmacy. Reports move through *Submitted*, *Under Review*, *Investigating* and then *Resolved* or *Rejected*, and resolution requires a note. Patients and caregivers can create one-time or recurring (daily, weekly, monthly) refill reminders, modify their time or recurrence, mark them done (which reschedules recurring reminders) or cancel them. All of these actions, like every administrative change, are written to the audit log.

5 Implementation

5.1 Technology Stack

The server runs on Node.js 22 [16] using only built-in modules: `node:http` for the server, `node:sqlite` for storage and `node:crypto` for hashing and token signing. The client uses no framework or build step. Avoiding third-party runtime dependencies removes supply-chain exposure and makes the system runnable with a single command (`npm start`). The code base comprises about 2,450 lines of server and client JavaScript and about 2,100 lines of tests.

5.2 Modularity

Following the information-hiding principle [17], the server is split into a 29-line composition root, a kernel, a request pipeline and ten component modules of 17–150 lines each that mirror the architecture diagram (Table 2). A pinned route table (40 routes) guards against accidental changes: a test fails if any route is added, removed, reordered or re-protected without updating the snapshot.

Table 2: Server component modules

Module	Responsibility
auth	Login, logout, current user, throttling of failed logins
catalogue	Catalogue search with filters, brand details, demo inventory
matching	Potential matches, comparison requests, pharmacist review, *My requests*
availability	Merged availability and prices; staff stock and price updates
reports	Incorrect-information reports and their review workflow
reminders	Create, modify, complete and cancel refill reminders
admin users	Create users, change roles, activate and deactivate accounts
admin catalogue	Hide, annotate, add brands, pack-size bulk import (CSV)
admin reports	Aggregated counts for the administrative dashboard
audit	Filtered, paged audit-log viewer

5.3 Data

The catalogue is the CDCI release used in the project (93,911 brands; 4,460 generics after exclusion). Pharmacy data is entirely synthetic: 20 fictional pharmacies in six Tamil Nadu cities with 875 inventory rows covering 120 brands. Every screen that shows this data carries a "DEMO DATA — NOT LIVE PHARMACY AVAILABILITY" label. Six demonstration accounts, one per role, are seeded on first start (Appendix A).

5.4 Source Code Repository

The complete project — source code, databases, configuration, tests, design documents and screenshots — is published at:

`https://github.com/bhuvaneshjatla-cpu/MedAccess`

6 Demonstration: How the System Works

This section walks through the working system role by role, using screenshots captured from the running application. Every screen shows the "Information only" banner, and every screen that displays pharmacy data is labelled as demo data.

6.1 Patient and Caregiver

Signing in and searching. Users sign in with an email address and password (Fig. 4); the navigation bar then shows only the pages their role may open. A patient searches by brand, generic or supplier name across 90,939 CDCI brands (Fig. 5).

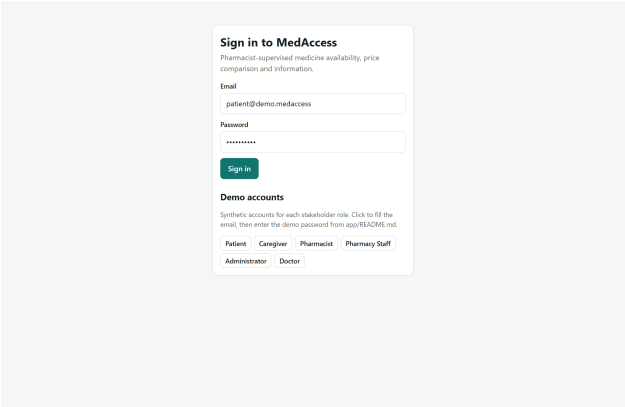


Figure 4: Login screen

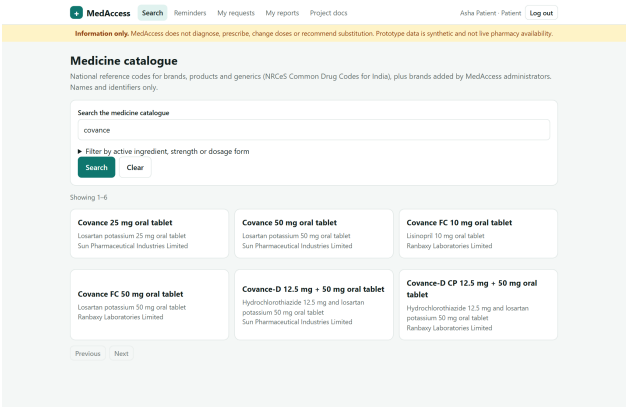


Figure 5: Catalogue search results for "covance"

Filters and restricted medicines. The filter panel narrows results by active ingredient, strength and dosage form: metformin, 500, tablet returns only 500 mg tablets (Fig. 6). Controlled and narcotic medicines are not in the index at all, so a search for tramadol returns nothing (Fig. 7).

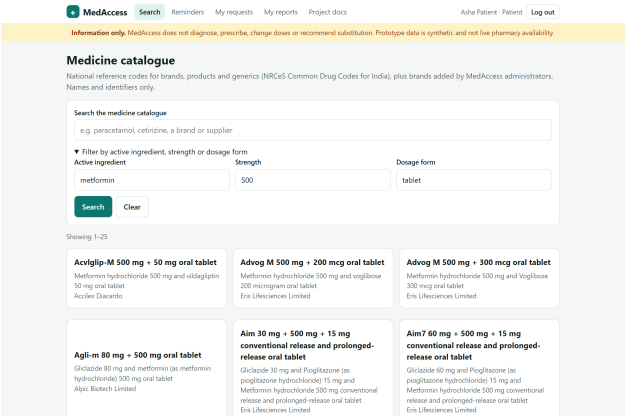


Figure 6: Search filtered by ingredient, strength and form

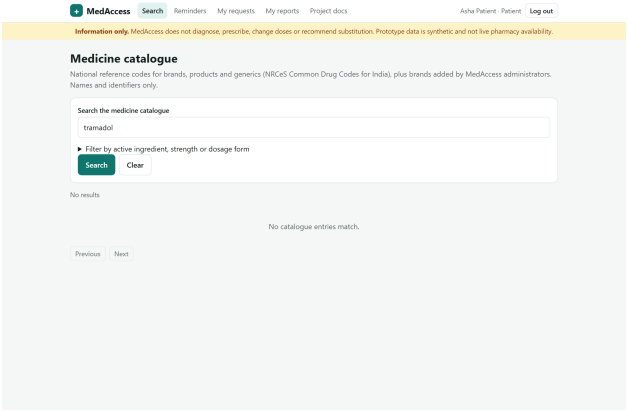


Figure 7: A restricted medicine (tramadol) cannot be found

Availability, prices and potential matches. Opening a brand shows the availability and indicative price at each pharmacy, each with its age and source; a row changed by pharmacy staff names the staff member (Fig. 8). Below it, other brands with the identical CDCI generic identifier are listed with what still differs — manufacturer, pack size and demo price — and the user may ask a pharmacist to verify any pair (Fig. 9).

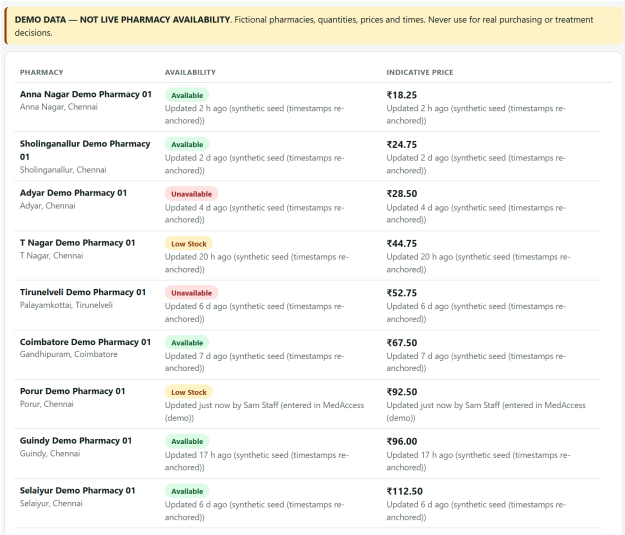


Figure 8: Availability and indicative prices with age and source

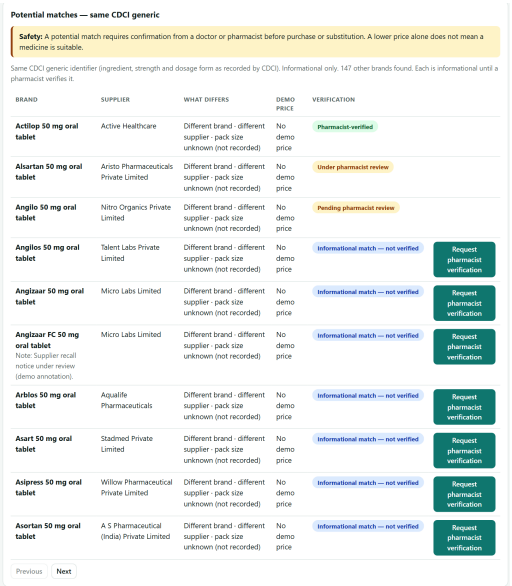


Figure 9: Potential matches and the verification request action

Requests, reminders and reports. The *My verification requests* page shows each requested comparison, its status, and the pharmacist's name and comment once decided (Fig. 10). Patients and caregivers can create one-time or recurring refill reminders and later edit their date or recurrence (Fig. 11). If a price or stock value is wrong, the user can report it and follow its review status.

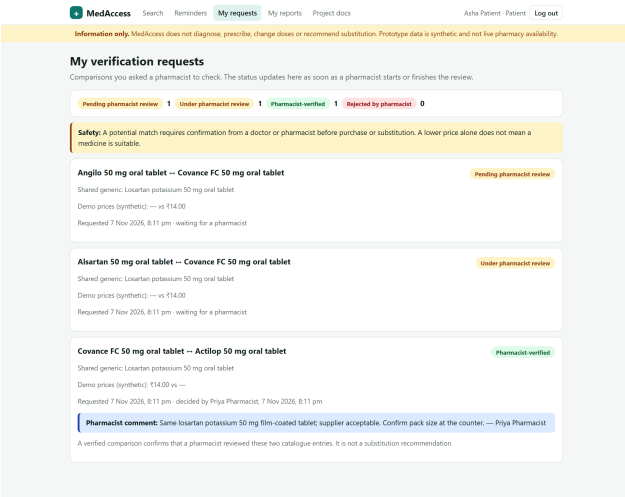


Figure 10: My verification requests

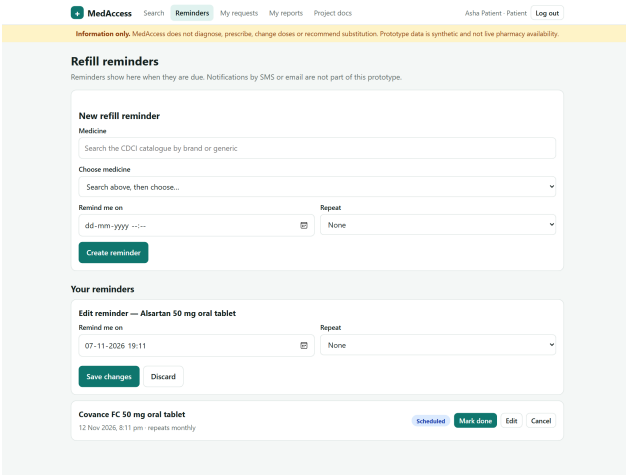


Figure 11: Refill reminders, editing a reminder

6.2 Pharmacist

The pharmacist's *Verification* page lists comparisons by status (Fig. 12). The review screen compares the two brands attribute by attribute — generic, strength, dosage form, pack size, supplier and demo price — and offers verify, reject and comment-only actions (Fig. 13). Verify and reject record the pharmacist's name, the time and the comment, which the requester then sees.

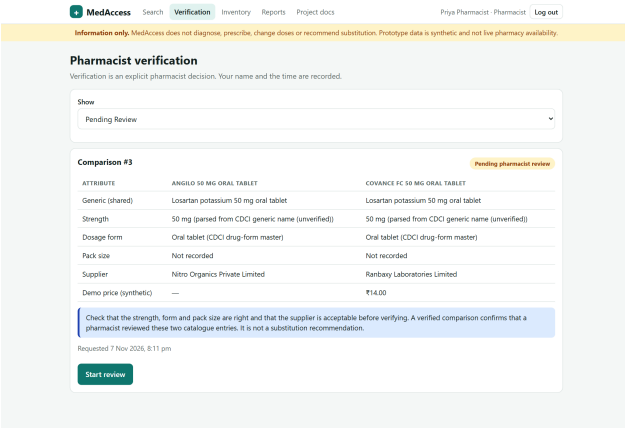


Figure 12: Pharmacist verification queue

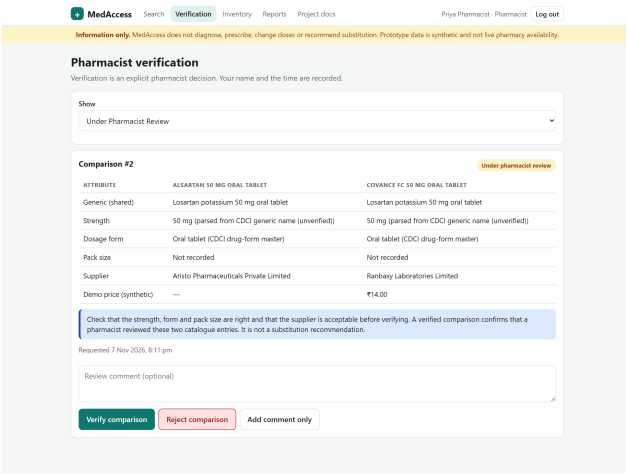


Figure 13: Attribute-by-attribute review with verify and reject

6.3 Pharmacy Staff

Staff choose a brand and update the availability status and price for their pharmacy directly in the availability table (Fig. 14). Each update is stored with the staff member's name and time, shown to patients and written to the audit log; the synthetic seed data is never overwritten. Staff and pharmacists also move incorrect-information reports through review and investigation to resolution, which requires a note (Fig. 15).

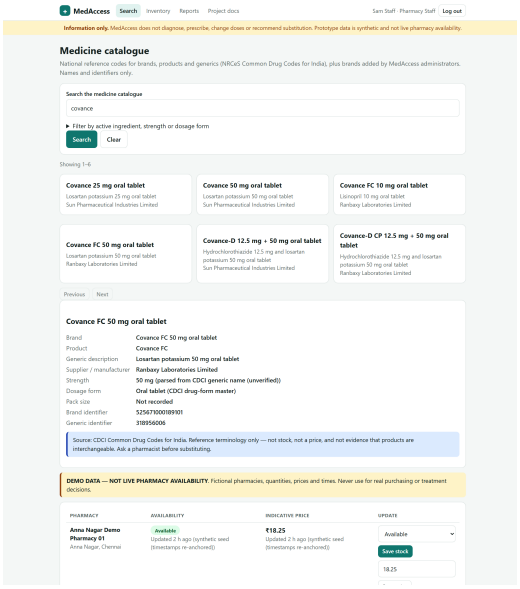


Figure 14: Stock and price editors (staff view)

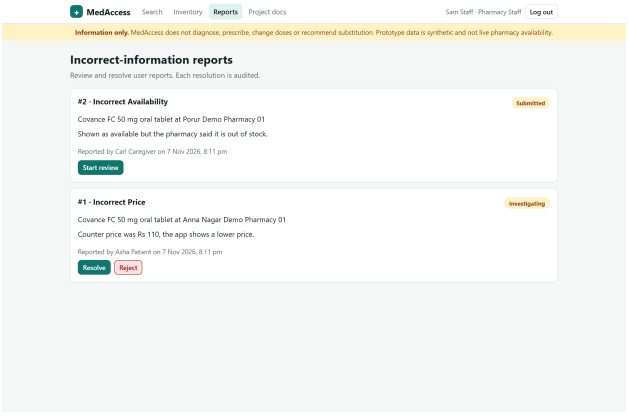


Figure 15: Reviewing incorrect-information reports

6.4 Administrator

Administrators create accounts, assign roles and activate or deactivate users; deactivation takes effect on the user's next request (Fig. 16). Because the CDCI catalogue is read-only, they correct it through an overlay: hide a brand, add a note, record pack size (individually or by CSV import with preview) or add a brand CDCI does not list (Fig. 17).

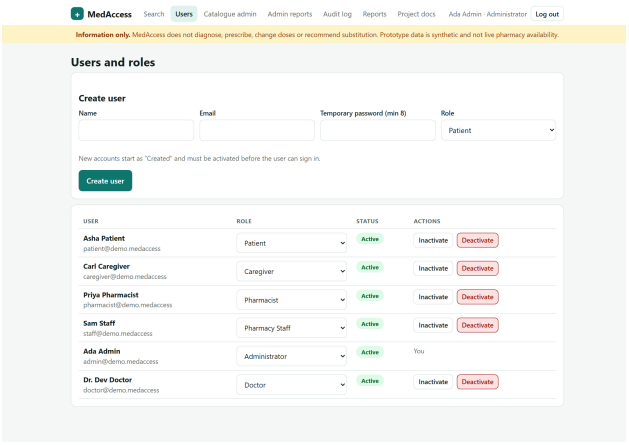


Figure 16: User and role management

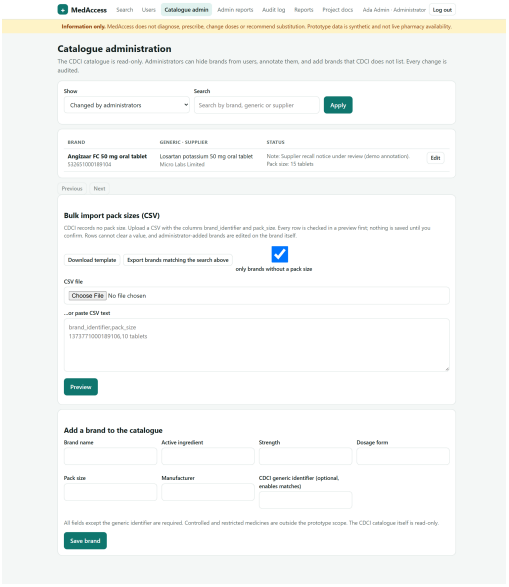


Figure 17: Catalogue administration

The reports dashboard summarises users, availability, comparisons, reports, reminders and stale data, and shows how many brands the restricted list excludes (Fig. 18). The audit log records every login, denied access and change with before and after values (Fig. 19).

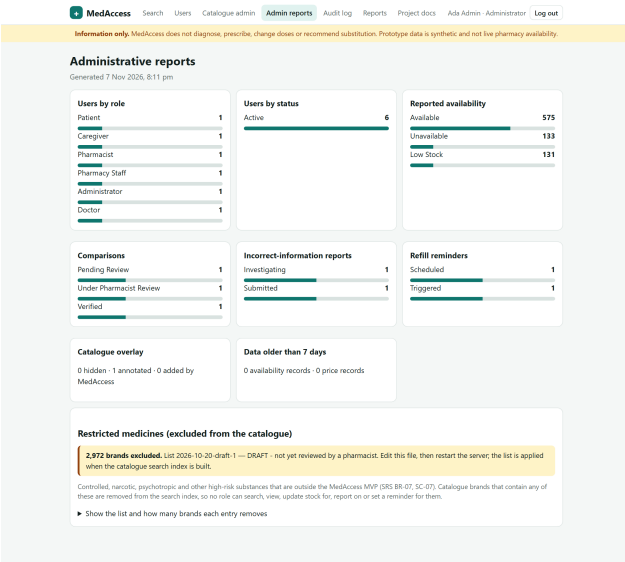


Figure 18: Administrative reports dashboard

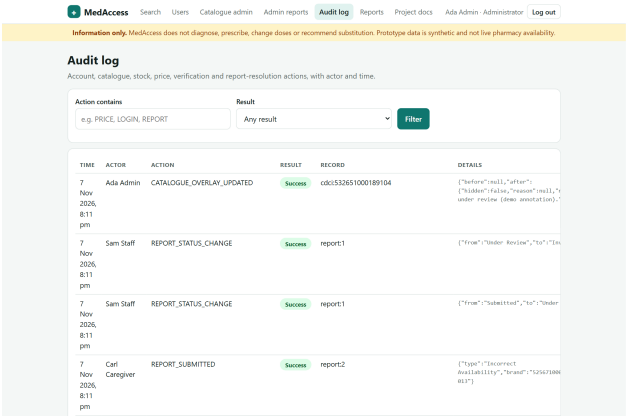


Figure 19: Audit log

6.5 Access Control, Mobile and Dark Mode

A doctor can search the catalogue and see potential matches but cannot open stock, price or administration pages; the page shows an access message and the server independently returns HTTP 403 (Fig. 20). The interface reflows down to 320 px wide and supports light and dark themes (Fig. 21).

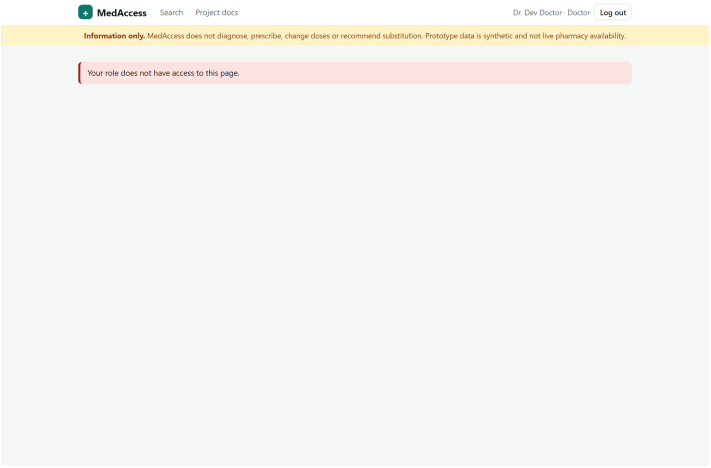


Figure 20: Access denied for a role without permission

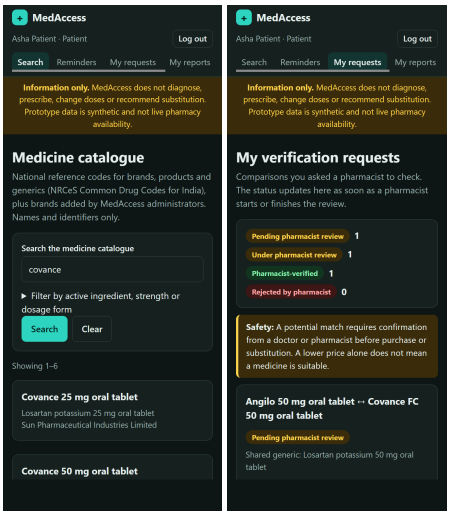


Figure 21: Mobile layout in dark mode

7 Testing and Evaluation

7.1 Test Methodology

Correctness was evaluated with 95 automated tests run by the Node.js test runner (Table 3). API tests start the real server on small fixture databases and assert status codes, role restrictions, state transitions and audit entries. Browser tests drive a headless Chromium-based browser through the DevTools protocol, sign in through the real login form and fail on any uncaught JavaScript error, console error or HTTP 5xx response. Accessibility tests audit each role's screens in light and dark themes for accessible names, headings, landmarks, table headers, keyboard reachability, target size and colour contrast. All 95 tests pass.

Table 3: Automated test suites

Suite	Focus	Tests
API	RBAC, verification workflow, audit, validation	6
Reference	CDCI catalogue, demo inventory, matching, re-anchoring	8
Parser	Strength parsing from generic names	11
Restricted	Exclusion of restricted medicines across every route	11
Filters	Ingredient, strength and form filters; synonyms	11
Overlay	Hide, annotate, pack size, added brands	7
CSV import	Pack-size bulk import with preview	16
Migration	Upgrade of older databases	2
Routes	Pinned route table; every route end to end	3
Browser	Real-browser flows for all roles, mobile	9
Accessibility	WCAG-oriented audit, light and dark, keyboard	11
Total		95

7.2 Performance

Latency was measured on a consumer laptop (Windows 11, Node.js 22.18) with the full catalogue loaded, issuing 50 sequential authenticated requests per endpoint. Table 4 reports the median and 95th percentile. Building the index at start-up took 4.8 s. Free-text search over 90,939 brands completes in 51–77 ms, compared with about 0.67 s for a single `LIKE` scan, and the match and availability endpoints respond in under 4 ms because they use indexed equality lookups.

Table 4: API latency (50 requests, full catalogue)

Endpoint	p50 (ms)	p95 (ms)
Search “losartan”	51.2	54.6
Search “paracetamol”	76.8	78.5
Filters: metformin, 500, tablet	33.3	34.6
Potential matches (10 per page)	2.4	3.9
Availability and prices for one brand	1.8	3.0

7.3 Safety Validation

The restricted list excluded 2,972 of 93,911 brands (3.2 %). Searches for tramadol, codeine and alprazolam returned no results, and the restricted-medicine suite verifies that excluded brands return *not found* through every route: search, details, availability, matches, stock and price updates, reports, reminders, the overlay and the raw demonstration inventory. A search for “morphine” returned only three apomorphine products, confirming that whole-word matching avoids excluding unrelated substances. Workflow tests confirm that no comparison reaches *Verified* without a pharmacist action, that non-pharmacist roles receive HTTP 403 on verification routes, and that each transition is audited.

7.4 Requirements Coverage

Each SRS requirement was traced to screens, routes and tests (Table 5). All 39 functional requirements are met or partly met. The five partial ones depend on data or decisions outside the prototype: pack size (FR-07, FR-16) is supported but must be entered by administrators because CDCI does not record it; CDCI entries can be hidden or annotated but not edited (FR-08, FR-36); and the required administrative reports (FR-37) were not specified in detail.

Table 5: SRS coverage of the implemented system

Category	Total	Met	Partly	Not testable
Functional (FR)	39	34	5	0
Non-functional (NFR)	18	14	1	3
Business rules (BR)	10	9	1	0

8 Discussion and Limitations

The central design choice of MedAccess is to treat matching as a decision-support aid rather than a recommendation. Restricting automatic matching to an identical CDCI generic identifier sacrifices recall (for example, it will not pair two salts of the same base), but every proposed match shares ingredients, strength and dosage form by construction, and the remaining differences are shown explicitly. The pharmacist workflow converts each comparison into an accountable professional judgement.

Several limitations remain:

- Availability and prices are synthetic, only 120 catalogue brands have demonstration inventory, and requirements were derived from simulated stakeholder responses, so the system has not been evaluated with live pharmacy data or real users; plain-language clarity and data-entry burden for pharmacy staff (NFR-10, NFR-12) therefore remain untested.
- The restricted-substance and synonym lists are drafts that require pharmacist review, and name-based matching can miss spellings the list does not contain.
- Strength is parsed from free-text names and labelled unverified.
- Reminders are shown in the application but are not delivered by SMS or e-mail.
- The prototype uses a single SQLite file and HTTP without TLS, and its audit log is not tamper-evident; production deployment would require TLS, a client–server database, persistent rate limiting and a hash-chained audit log.

9 Conclusion and Future Work

MedAccess shows that a national drug terminology can support useful availability, price and alternative-brand information for patients without allowing the software to make substitution decisions. Matching is constrained to identical CDCI generics, verification is reserved for pharmacists through an audited state machine, controlled substances are excluded before any query can reach them, and every data point carries its source and age. The system meets or partly meets all functional requirements of its specification, passes 95 automated tests, and answers catalogue searches in under 80 ms.

Future work includes integration with real pharmacy inventory feeds, pharmacist review of the restricted and synonym lists, structured pack-size data from manufacturers, reminder delivery through SMS or e-mail, a “request more information” step in the verification workflow, and a field study with patients, caregivers and pharmacists to evaluate usability and the effect of verified comparisons on out-of-pocket spending.

A Running the System and Demo Accounts

A.1 Download

The project is available at `https://github.com/bhuvaneshjatla-cpu/MedAccess`. Use the green **Code** button and **Download ZIP**, then unzip it, or clone it with:

```
git clone https://github.com/bhuvaneshjatla-cpu/MedAccess.git
```

A.2 Run

1. Install **Node.js 22.13 or newer** from `https://nodejs.org`. No other software and no `npm install` step is required.
2. Open a terminal in the `app` folder of the project and run:

   ```
   npm start
   ```

3. Open `http://localhost:3000` in a web browser. The first start takes a few seconds while the catalogue index is built.
4. Optional: `npm test` runs the 95 automated tests, and `npm run reset-db` restores the demo data.

A.3 Demo Accounts

All accounts use the password `Demo@12345`.

Table 6: Demo accounts

Role	Email	What to try
Patient	`patient@demo.medaccess`	Search *losartan*, open a brand, view availability, prices and potential matches, ask a pharmacist, *My requests*, reminders, report wrong information
Caregiver	`caregiver@demo.medaccess`	Same as the patient, on behalf of a family member
Pharmacist	`pharmacist@demo.medaccess`	*Verification* queue: start review, verify or reject with a comment; update stock and price
Pharmacy Staff	`staff@demo.medaccess`	*Inventory*: update stock status and price; review incorrect-information reports
Administrator	`admin@demo.medaccess`	Users and roles, catalogue administration, reports dashboard, audit log
Doctor	`doctor@demo.medaccess`	Catalogue search and potential matches (no stock or price access)

All pharmacy, stock and price data is synthetic demonstration data and must not be used for real purchasing or treatment decisions.

References

[1] A. Cameron, M. Ewen, D. Ross-Degnan, D. Ball, and R. Laing, "Medicine prices, availability, and affordability in 36 developing and middle-income countries: A secondary analysis," *The Lancet*, vol. 373, no. 9659, pp. 240–249, Jan. 2009.

[2] World Health Organization and Health Action International, *Measuring Medicine Prices, Availability, Affordability and Price Components*, 2nd ed. Geneva, Switzerland: WHO, 2008.

[3] Pharmaceuticals and Medical Devices Bureau of India, "Pradhan Mantri Bhartiya Janaushadhi Pariyojana (PMBJP)." [Online]. Available: `https://janaushadhi.gov.in`

[4] National Resource Centre for EHR Standards (NRCeS), C-DAC Pune, "Common Drug Codes for India (CDCI)." [Online]. Available: `https://www.nrces.in`

[5] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996.

[6] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," *ACM Trans. Inf. Syst. Secur.*, vol. 4, no. 3, pp. 224–274, Aug. 2001.

[7] OWASP Foundation, "OWASP Top 10:2021." [Online]. Available: `https://owasp.org/Top10/`

[8] J. Walonoski *et al.*, "Synthea: An approach, method, and software mechanism for generating synthetic patients and the synthetic electronic health care record," *J. Amer. Med. Inform. Assoc.*, vol. 25, no. 3, pp. 230–238, Mar. 2018.

[9] ISO/IEC 29148:2018, *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, 2018.

[10] Government of India, *The Narcotic Drugs and Psychotropic Substances Act*, Act No. 61 of 1985.

[11] W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, Jun. 2018. [Online]. Available: `https://www.w3.org/TR/WCAG21/`

[12] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," IETF RFC 2104, Feb. 1997.

[13] C. Percival, "Stronger key derivation via sequential memory-hard functions," presented at BSDCan, Ottawa, Canada, May 2009.

[14] D. R. Hipp, "SQLite." [Online]. Available: `https://www.sqlite.org`

[15] D. Harel, "Statecharts: A visual formalism for complex systems," *Sci. Comput. Program.*, vol. 8, no. 3, pp. 231–274, Jun. 1987.

[16] OpenJS Foundation, "Node.js." [Online]. Available: `https://nodejs.org`

[17] D. L. Parnas, "On the criteria to be used in decomposing systems into modules," *Commun. ACM*, vol. 15, no. 12, pp. 1053–1058, Dec. 1972.